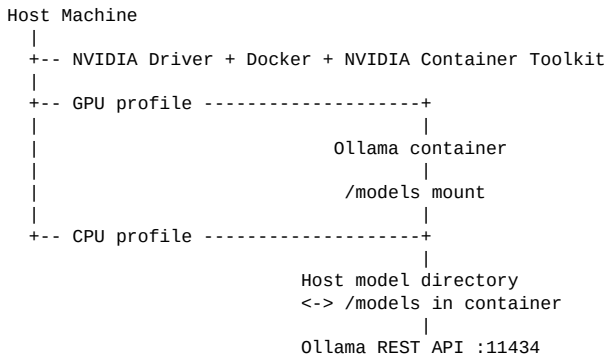


Enterprise-Grade Ollama Local AI Deployment

GPU + CPU | Bidirectional Persistent Model Storage | Docker Security | RTX 5080 Performance Testing

Purpose: A portable, security-conscious local Ollama architecture with one persistent host model location, GPU/CPU deployment modes, environment-driven configuration, and reproducible concurrency benchmarking.

1. Architecture



2. Recommended Project Structure

```
ollama-enterprise/
├── .env
├── .env.example
├── .gitignore
├── compose.gpu.yml
├── compose.cpu.yml
├── Dockerfile
├── Dockerfile.cpu
├── config/
│   ├── models.txt
│   └── scripts/
│       ├── start-gpu.sh
│       ├── start-cpu.sh
│       ├── pull-models.sh
│       └── benchmark.sh
├── benchmark/
│   ├── benchmark.py
│   ├── requirements.txt
│   └── results/
├── monitoring/
│   ├── prometheus.yml
│   └── data/
│       └── logs/
```

3. .env — Single Configuration Source

```
# Application
OLLAMA_IMAGE=ollama/ollama
OLLAMA_VERSION=latest
OLLAMA_CONTAINER_NAME=ollama
OLLAMA_HOST=0.0.0.0:11434
OLLAMA_PORT=11434

# Persistent model storage
OLLAMA_MODELS_HOST=/opt/ai/ollama/models
OLLAMA_MODELS_CONTAINER=/models

# Serving
OLLAMA_NUM_PARALLEL=2
OLLAMA_MAX_LOADED_MODELS=1
OLLAMA_MAX_QUEUE=128
OLLAMA_KEEP_ALIVE=10m
OLLAMA_CONTEXT_LENGTH=8192
OLLAMA_LOAD_TIMEOUT=10m

# Local/private operation
OLLAMA_NO_CLOUD=true

# NVIDIA
NVIDIA_VISIBLE_DEVICES=0
NVIDIA_DRIVER_CAPABILITIES=compute,utility

# Docker resources
CPU_LIMIT=16
```

```

MEMORY_LIMIT=24g
CPU_RESERVATION=4
MEMORY_RESERVATION=8g

# Network
OLLAMA_NETWORK=ollama_net

# Logging
LOG_MAX_SIZE=10m
LOG_MAX_FILES=5

# Benchmark
BENCHMARK_MODEL=qwen3:8b
BENCHMARK_USERS=1,2,4,8,12,16
BENCHMARK_REQUESTS=50

```

4. .env.example

Commit the example configuration; keep the real .env out of source control.

```

OLLAMA_IMAGE=ollama/ollama
OLLAMA_VERSION=latest
OLLAMA_CONTAINER_NAME=ollama
OLLAMA_HOST=0.0.0.0:11434
OLLAMA_PORT=11434

OLLAMA_MODELS_HOST=/opt/ai/ollama/models
OLLAMA_MODELS_CONTAINER=/models

OLLAMA_NUM_PARALLEL=2
OLLAMA_MAX_LOADED_MODELS=1
OLLAMA_MAX_QUEUE=128
OLLAMA_KEEP_ALIVE=10m
OLLAMA_CONTEXT_LENGTH=8192
OLLAMA_LOAD_TIMEOUT=10m
OLLAMA_NO_CLOUD=true

NVIDIA_VISIBLE_DEVICES=0
NVIDIA_DRIVER_CAPABILITIES=compute,utility

CPU_LIMIT=16
MEMORY_LIMIT=24g
CPU_RESERVATION=4
MEMORY_RESERVATION=8g

OLLAMA_NETWORK=ollama_net
LOG_MAX_SIZE=10m
LOG_MAX_FILES=5

```

5. Dockerfiles

Use the official Ollama image rather than rebuilding the Ollama/CUDA runtime unnecessarily. GPU access is supplied by the NVIDIA container runtime.

```

FROM ollama/ollama:latest

USER root

RUN mkdir -p /models && chmod 755 /models

ENV OLLAMA_MODELS=/models

EXPOSE 11434

HEALTHCHECK --interval=30s --timeout=5s --start-period=20s --retries=3 \
  CMD ollama list >/dev/null 2>&1 || exit 1

ENTRYPOINT ["ollama"]
CMD ["serve"]

```

Use the same Dockerfile for CPU if desired; the GPU/CPU difference is primarily the runtime configuration. A separate Dockerfile.cpu can duplicate this file for explicit deployment separation.

6. GPU Compose

```

services:
  ollama:
    build:
      context: .
      dockerfile: Dockerfile

    container_name: ${OLLAMA_CONTAINER_NAME}
    restart: unless-stopped

```

```

ports:
  - "127.0.0.1:${OLLAMA_PORT}:11434"

environment:
  OLLAMA_HOST: ${OLLAMA_HOST}
  OLLAMA_MODELS: ${OLLAMA_MODELS_CONTAINER}
  OLLAMA_NUM_PARALLEL: ${OLLAMA_NUM_PARALLEL}
  OLLAMA_MAX_LOADED_MODELS: ${OLLAMA_MAX_LOADED_MODELS}
  OLLAMA_MAX_QUEUE: ${OLLAMA_MAX_QUEUE}
  OLLAMA_KEEP_ALIVE: ${OLLAMA_KEEP_ALIVE}
  OLLAMA_CONTEXT_LENGTH: ${OLLAMA_CONTEXT_LENGTH}
  OLLAMA_LOAD_TIMEOUT: ${OLLAMA_LOAD_TIMEOUT}
  OLLAMA_NO_CLOUD: ${OLLAMA_NO_CLOUD}
  NVIDIA_VISIBLE_DEVICES: ${NVIDIA_VISIBLE_DEVICES}
  NVIDIA_DRIVER_CAPABILITIES: ${NVIDIA_DRIVER_CAPABILITIES}

volumes:
  - type: bind
    source: ${OLLAMA_MODELS_HOST}
    target: ${OLLAMA_MODELS_CONTAINER}

deploy:
  resources:
    limits:
      cpus: "${CPU_LIMIT}"
      memory: ${MEMORY_LIMIT}
    reservations:
      cpus: "${CPU_RESERVATION}"
      memory: ${MEMORY_RESERVATION}
    devices:
      - driver: nvidia
        count: 1
        capabilities: [gpu]

security_opt:
  - no-new-privileges:true

cap_drop:
  - ALL

networks:
  - ollama_net

logging:
  driver: json-file
  options:
    max-size: ${LOG_MAX_SIZE}
    max-file: ${LOG_MAX_FILES}

networks:
  ollama_net:
    name: ${OLLAMA_NETWORK}
    driver: bridge

```

7. CPU Compose

```

services:
  ollama:
    build:
      context: .
      dockerfile: Dockerfile.cpu

  container_name: ${OLLAMA_CONTAINER_NAME}
  restart: unless-stopped

ports:
  - "127.0.0.1:${OLLAMA_PORT}:11434"

environment:
  OLLAMA_HOST: ${OLLAMA_HOST}
  OLLAMA_MODELS: ${OLLAMA_MODELS_CONTAINER}
  OLLAMA_NUM_PARALLEL: ${OLLAMA_NUM_PARALLEL}
  OLLAMA_MAX_LOADED_MODELS: ${OLLAMA_MAX_LOADED_MODELS}
  OLLAMA_MAX_QUEUE: ${OLLAMA_MAX_QUEUE}
  OLLAMA_KEEP_ALIVE: ${OLLAMA_KEEP_ALIVE}
  OLLAMA_CONTEXT_LENGTH: ${OLLAMA_CONTEXT_LENGTH}
  OLLAMA_LOAD_TIMEOUT: ${OLLAMA_LOAD_TIMEOUT}
  OLLAMA_NO_CLOUD: ${OLLAMA_NO_CLOUD}

volumes:
  - type: bind
    source: ${OLLAMA_MODELS_HOST}
    target: ${OLLAMA_MODELS_CONTAINER}

deploy:
  resources:
    limits:
      cpus: "${CPU_LIMIT}"

```

```

        memory: ${MEMORY_LIMIT}
    reservations:
        cpus: "${CPU_RESERVATION}"
        memory: ${MEMORY_RESERVATION}

    security_opt:
        - no-new-privileges:true

    cap_drop:
        - ALL

    networks:
        - ollama_net

    logging:
        driver: json-file
        options:
            max-size: ${LOG_MAX_SIZE}
            max-file: ${LOG_MAX_FILES}

networks:
    ollama_net:
        name: ${OLLAMA_NETWORK}
        driver: bridge

```

8. Model Storage — Bidirectional Persistent Mount

```

Host:
    /opt/ai/ollama/models

Container:
    /models

Environment:
    OLLAMA_MODELS=/models

Flow:
    ollama pull <model>
    ↓
    container /models
    ↑ bind mount
    host /opt/ai/ollama/models

```

Container deletion does NOT delete the host model repository.

Windows example

```

OLLAMA_MODELS_HOST=D:/AI/ollama/models
OLLAMA_MODELS_CONTAINER=/models

```

9. Deployment Commands

```

# GPU
docker compose -f compose.gpu.yml --env-file .env up -d --build

# CPU
docker compose -f compose.cpu.yml --env-file .env up -d --build

# Status
docker compose ps

# API
curl http://127.0.0.1:11434/api/tags

# Models
docker exec ollama ollama list

# Pull a model
docker exec ollama ollama pull qwen3:8b

# Stop
docker compose down

```

10. GPU Validation

```

docker run --rm --gpus all \
    nvidia/cuda:12.9.1-base-ubuntu24.04 nvidia-smi

nvidia-smi

docker exec ollama ollama list

```

Validate the NVIDIA driver and container runtime first. If nvidia-smi does not work in a GPU container, fix the NVIDIA Container Toolkit/runtime before troubleshooting Ollama.

11. Security Baseline

Control	Recommended practice
Network	Bind host port to 127.0.0.1 unless LAN access is explicitly required.
GPU	Expose only the required GPU; avoid NVIDIA_VISIBLE_DEVICES=all.
Cloud	Set OLLAMA_NO_CLOUD=true for a private local-only deployment.
Privileges	no-new-privileges + cap_drop: ALL.
Docker socket	Never mount /var/run/docker.sock into Ollama.
Secrets	Never put credentials/API keys in Dockerfiles or images.
Filesystem	Mount only the model repository and required data.
Persistence	Use host bind mount for models; never rely on container writable storage.
Updates	Pin tested image versions for production/reproducible environments.

12. RTX 5080 Performance Strategy

Do not equate concurrent users with Ollama parallelism. For example, 16 clients can be served through a queue while Ollama runs a smaller number of simultaneous generations. Higher parallelism also increases memory pressure because context must be maintained for multiple requests.

Profile	Parallel	Context	Use
Latency	1	8K	Interactive coding/chat
Balanced	2	8K	General local AI serving
Throughput	4	8K	Benchmark candidate; validate VRAM/TPS

13. Concurrency Test Matrix

Test	Users	Parallel	Context	Requests
T01	1	1	4K	50
T02	2	1	4K	50
T03	4	2	4K	100
T04	8	2	4K	100
T05	12	2	4K	120
T06	16	2	4K	160
T07	4	4	8K	100
T08	8	4	8K	160
T09	12	4	8K	240
T10	16	4	8K	320

14. Metrics to Capture

Application:
TTFT (time to first token)
End-to-end latency
P50 / P90 / P95 / P99
Input tokens/sec
Output tokens/sec
Requests/sec
Error rate
Queue time

System:
GPU utilization
GPU memory used / total
GPU temperature
GPU power
CPU utilization
RAM utilization

```
Run two modes:
Cold model: model must load
Warm model: model already resident
```

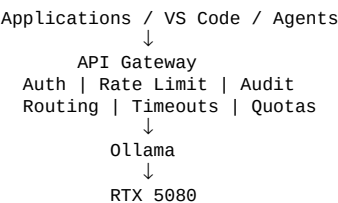
15. GPU Telemetry

```
nvidia-smi \
--query-gpu=timestamp,name,utilization.gpu,utilization.memory,memory.used,memory.total,temperature.gpu,power.draw \
--format=csv
```

16. Model Benchmark Matrix

Model class	Quantization	Context	Users
7B–8B	Q4/Q5/Q8	4K/8K	1/4/8/16
14B	Q4/Q5	4K/8K	1/4/8/16
27B–32B	Q4	4K/8K	1/4/8/16

17. Enterprise API Layer



For multiple applications or users, keep Ollama as the inference engine and place authentication, rate limiting, request IDs, quotas, routing, and audit logging in a gateway layer.

18. Recommended Baseline for RTX 5080

```
OLLAMA_NUM_PARALLEL=2
OLLAMA_MAX_LOADED_MODELS=1
OLLAMA_MAX_QUEUE=128
OLLAMA_KEEP_ALIVE=10m
OLLAMA_CONTEXT_LENGTH=8192
OLLAMA_NO_CLOUD=true
NVIDIA_VISIBLE_DEVICES=0
```

Start with parallel=2.
Benchmark 1, 2, and 4.
Do not assume 16 concurrent users requires parallel=16.
Choose the final value from measured P95 latency,
output TPS, VRAM usage, and error rate.

19. Production Checklist

- Persistent model directory is on the host.
- Real .env is excluded from Git.
- GPU visibility is restricted to the required device.
- Ollama is bound to localhost unless LAN exposure is intentional.
- Cloud features are disabled for private/local workloads.
- Container capabilities are dropped and no-new-privileges is enabled.
- No Docker socket is mounted.
- Health checks are enabled.
- Docker log rotation is configured.
- Cold and warm model tests are separated.
- Concurrency is tested at 1, 2, 4, 8, 12 and 16 users.
- P50/P95/P99, TTFT, output TPS, VRAM and GPU utilization are recorded.

- The final parallelism setting is based on measured results.
- Model/image versions are pinned after validation for reproducibility.

Reference notes

Ollama configuration references: OLLAMA_MODELS, OLLAMA_NUM_PARALLEL, OLLAMA_MAX_LOADED_MODELS, OLLAMA_MAX_QUEUE, OLLAMA_KEEP_ALIVE, OLLAMA_CONTEXT_LENGTH and OLLAMA_NO_CLOUD are documented in Ollama's environment configuration. NVIDIA Container Toolkit provides Docker GPU access and supports restricting visible GPUs. Verify the exact options against the Ollama and NVIDIA documentation for the versions you deploy.